

REPORT 2:

Online Advertisement Ranking Using Quick Sort

Title:

Online Advertisement Ranking Using Quick Sort

Objective

To analyze the use of Quick Sort for ranking advertisements based on relevance score and bid value in real-time advertisement platforms.

Problem Statement

Online advertising systems continuously rank advertisements according to advertiser bids and user relevance scores. The ranking process must be fast and efficient to support real-time ad delivery.

Algorithm Used

Quick Sort

Quick Sort follows the Divide and Conquer strategy:

1. Select a pivot element.
2. Partition data into smaller and larger elements.
3. Recursively sort partitions.
4. Combine the sorted partitions.

Implementation

Advertisement ranking scores are sorted using Quick Sort to generate the final advertisement display order.

Working Process

1. Select a pivot advertisement score.
2. Partition scores into lower and higher groups.
3. Recursively sort each group.
4. Combine results to obtain ranked advertisements.

Issues Identified

Pivot Selection

Choosing a poor pivot can create highly unbalanced partitions and reduce performance.

Data Distribution

Nearly sorted data may lead to worst-case behavior if improper pivots are selected.

Duplicate Values

Advertisements may have identical relevance scores. Three-way partitioning can efficiently handle duplicates.

Optimizations

Random Pivot Selection

Randomly selecting pivots reduces the probability of worst-case performance.

Median-of-Three Pivot

Selecting the median of first, middle, and last elements improves partition balance.

Three-Way Partitioning

Separates elements into:

- Less than pivot
- Equal to pivot
- Greater than pivot

This improves performance when many duplicate values exist.

Time Complexity Analysis

Case	Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n^2)$

Space Complexity

$O(\log n)$

The screenshot shows a VS Code editor with a Python file named 'Online Advertisement Ranking using Qui.py'. The code implements a quicksort algorithm to rank advertisements based on their 'rank_score'. The dataset includes: Laptop Sale (85), Mobile Offer (95), Online Course (78), Insurance Plan (92), and Travel Package (88). The terminal output shows the sorted list of advertisements in descending order of rank score.

```

1  # Online Advertisement Ranking using Quick Sort
2
3  def quick_sort(arr):
4      if len(arr) <= 1:
5          return arr
6
7      pivot = arr[len(arr) // 2]["rank_score"]
8
9      left = [x for x in arr if x["rank_score"] > pivot]
10     equal = [x for x in arr if x["rank_score"] == pivot]
11     right = [x for x in arr if x["rank_score"] < pivot]
12
13     return quick_sort(left) + equal + quick_sort(right)
14
15 # Advertisement dataset
16 ads = [
17     {"ad": "Laptop Sale", "rank_score": 85},
18     {"ad": "Mobile Offer", "rank_score": 95},
19     {"ad": "Online Course", "rank_score": 78},
20     {"ad": "Insurance Plan", "rank_score": 92},
21     {"ad": "Travel Package", "rank_score": 88},
22 ]
23
24 def main():
25     # Sort the advertisements
26     sorted_ads = quick_sort(ads)
27
28     # Print the sorted advertisements
29     for ad in sorted_ads:
30         print(ad)
31
32 if __name__ == '__main__':
33     main()

```

```

PS C:\Users\lenovo\OneDrive\Documents> c:\cd 'c:\Users\lenovo\OneDrive\Documents'; & 'C:\Users\lenovo\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\lenovo\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '58227' '--' 'c:\Users\lenovo\OneDrive\Documents\Online Advertisement Ranking using Qui.py'
Ranked Advertisements:
{'ad': 'Mobile Offer', 'rank_score': 95}
{'ad': 'Insurance Plan', 'rank_score': 92}
{'ad': 'Travel Package', 'rank_score': 88}
{'ad': 'Laptop Sale', 'rank_score': 85}
{'ad': 'Online Course', 'rank_score': 78}

```

Advantages

- Fast average-case performance.
- Low memory usage.
- In-place sorting.
- Suitable for real-time systems.
- Efficient cache utilization.

Result

Advertisements were successfully ranked according to relevance scores and bid values. The ranking process was completed efficiently, supporting quick advertisement delivery.

Conclusion

Quick Sort is an effective choice for online advertisement ranking systems due to its excellent average-case performance and low memory requirements. With proper pivot selection techniques, Quick Sort provides efficient real-time ranking for large advertisement datasets.